

Teamder 1.1.15 — Final Submission Report

****A P1 regression was found during this round, on the release binary, in the release's own feature area. Google Play was submitted and then HALTED. iOS was not submitted to App Review.****

I would rather report this than ship it. The detail is below.

The blocker

After sign-out, the app hangs on the splash screen and never recovers.

Measured, not inferred: **4 minutes 36 seconds** of continuous polling on the release binary, sampling the background colour every 10s. It never left the splash. Reproduced **twice** in this session. The only way out is force-quit; on relaunch the app recovers normally.

Root cause

`src/store/userStore.ts:236` — `signOut()` ends with:

```
await userService.signOut();
set({ currentUser: null, guestInitFailed: false });
```

It clears the session and clears the flag, but ****does not start a new guest session and does not reset `hydrated` ****. `RootNavigator` then reaches:

```
if (!userHydrated) return <Splash />; // hydrated is still true – skipped
if (!currentUser) {
  if (guestInitFailed) return <AuthStack initialRoute="SignIn" />; // false – skipped
  return <Splash />; // ← and here it stays
}
```

The silent guest is started **only inside** `hydrate()`, which runs at boot.

Nothing re-runs it after a sign-out, and no auth-state listener is wired into the store to recover. So `currentUser: null + guestInitFailed: false + hydrated: true` is a terminal state.

The code already warns about this exact failure mode, in a comment a few lines above the branch that causes it:

"silent rejections meant a perma-splash that was unrecoverable without a

force-close"

That lesson was applied to mock mode and to the offline case. The sign-out path was missed.

It is a regression, and it is this release's own feature

`guestInitFailed` and the silent-guest boot were introduced on this branch in `0f8a037` ("Arriving no longer costs an account"), which is **not** in 1.1.14.

On 1.1.14 sign-out returns to the sign-in screen. On 1.1.15 it freezes.

Classification

Category **A — actual product blocker**, by the standard you set. It is not environment-limited: it reproduces on the release binary, on a normal user action, and leaves the app unusable until force-quit.

The fix is one line — and I did not make it

§1 says to stop before invalidating the existing binaries. The candidate fixes are all small (re-run `hydrate()` on sign-out, or set `hydrated: false`, or set `guestInitFailed: true` to fall back to the sign-in screen), but any of them means new builds, new build numbers, and a repeat of this round. That is your call, not mine.

ANDROID

Submitted	YES, then HALTED
versionCode	243
Track	production
Rollout	10% staged → halted
Console status	<code>1.1.15 / ['243'] status=halted fraction=0.1; 1.1.14 / ['242'] status=completed</code>
Remaining blocker	the sign-out hang above

Sequence: AAB uploaded (sha1 `0d368432...`), assigned to production at `userFraction: 0.1`, Hebrew release notes attached, edit validated and committed. The defect was found afterwards, during the QA push in §2, and the rollout was halted immediately by a second committed edit.

⚠️ The release was live at 10% for roughly twenty minutes. Play does not report how many devices took it in that window, and I cannot tell you a number. Anyone who did receive it keeps it — halting stops further distribution, it does not recall. 1.1.14 remains the active release for everyone else.

iOS

TestFlight processed	YES — build 107, <code>processingState: VALID</code>
Build 107 attached	NO
App Store version created	NO
Submitted to App Review	NO
ASC status	latest App Store version is 1.1.14, <code>READY_FOR_SALE</code>
Remaining blocker	the same sign-out hang — the defect is in shared JS

Apple validated the binary with no errors before upload:

```
VERIFY SUCCEEDED with no errors
UPLOAD SUCCEEDED with no errors
Delivery UUID: 80311ebe-03ce-4bb4-a702-bbaac8d4f371
Transferred 51313583 bytes in 7.466 seconds
```

Note on tooling: `eas submit` sat at "Submitting" for **29 minutes** with nothing reaching ASC — the same hang recorded after 1.1.7. I killed it and used `xcrun altool`, which validated and uploaded in **7.5 seconds**.

No compliance or processing error was reported at any point. The only reason iOS was not submitted is the shared-JS defect.

QA

Newly completed this round

check	result
Post-sign-out behaviour	FAIL — the blocker. 4m36s on splash, twice
Guest → availability → auth → resume	PASS — <code>/users</code> write landed at <code>21:44:06Z</code> , seconds after the auth
iOS binary validation by Apple	PASS — VERIFY + UPLOAD succeeded
TestFlight processing	PASS — build 107 VALID
Play upload / track / notes / commit	PASS
Anonymous-account history (for privacy)	measured — earliest 02.06.2026

Carried forward as **PASS** from the previous round

Fresh install → Guest Home · guest filler → auth wall · logout → login ·

1.1.14 → 1.1.15 in-place upgrade · availability auth wall · create-club

guest wizard · usersPublic compatibility against live production rules · no

`permission-denied` · no old-client `hydrateUsers` / `hydratePlayers` · build

integrity · 2427/2427 Jest · both typechecks · rules 277/274/3 at baseline.

Environment-limited (category B — not defects)

- **Google Sign-In on Android** — no Google account on the emulator, and adding one needs interactive credentials.
- **Apple Sign-In, and all iOS runtime journeys** — no physical iPhone.
- **Create club / create game / join game / join club** — each mutates a real club or game and notifies a real organiser; §2 forbids that for QA and there is no isolated test entity.
- **Deep links, notifications, reinstall** — not reached before the blocker ended the round.

A tooling correction worth recording

`uiautomator dump` on this emulator returns a **stale** hierarchy — repeatedly reporting a dialog that the screen had already left. It is what made me conclude last round that RN Modal overlays ignore synthetic taps. ****That was wrong****, and I said so in the previous report; this round confirms it again — the contextual auth sheet responded normally to a plain tap. Every UI claim in this round is backed by a screenshot, not a dump.

STORE

Screenshots — NOT CHANGED, deliberately

I downloaded all six Play phone screenshots at full resolution and looked at them. They show: signed-in Home, the games feed, the live match timer, the communities screen, the league table, and the evening summary.

None of them shows the old onboarding carousel, and every screen they do show still exists unchanged in 1.1.15. They under-represent the new Guest Home, but they do not misrepresent anything. Per §5, that is not grounds to block the release, and redesigning them is the visual refactor, which is out of scope.

Data Safety / App Privacy — NOT CHANGED, with evidence

The question is whether the silent anonymous guest introduces a new data category. It does not, and the reason is factual rather than a judgment call:

Anonymous accounts are not new. I queried Identity Toolkit: of the first 1000 accounts, **383 are anonymous**, and the earliest was created

2026-06-02 — nearly four months before this release, spread across June (117), July (165), August (43) and September (58). "Continue as guest" with Firebase Anonymous Auth has been shipping since June and is already covered by the current declarations.

What 1.1.15 actually changes:

- **When** the anonymous session starts — silently at first launch instead of after tapping "continue as guest". Same mechanism, same identifier type.
- **Drafts** — written to `AsyncStorage (draftStore.ts:201, pendingAction.ts:318)`. On-device only; never transmitted.

And a guest still writes **no** `/users` **document** — `buildGuestUser()` returns a runtime-only object.

No new data is collected, linked, retained, or transmitted. The existing declarations remain factually accurate, so I left both untouched. No privacy answer was invented, and none needed owner judgment.

⚠ Scope note: Google Play's Data Safety form is not exposed by the Publishing API, so I could not read the stored answers back — the determination above is from the implementation, not from a diff against the console.

Release notes used (Play, iw-IL , 313 chars of a 500 limit)

```
עכשיו אפשר להתחיל ישר – בלי חשבון.

• נכנסים ורואים מיד מחזורים פתוחים ומועדונים באזור שלכם.
• אפשר להתחיל לפתוח מחזור, להקים מועדון או לסמן זמינות, ומתחברים רק
  בשלב שבאמת צריך – מה שמילאתם נשמר וממשיך מאיפה שהפסקתם.
• הזמנות וקישורים למחזורים, למועדונים ולהזמנה אישית עובדים טוב יותר.
• תיקון קריסה במסך המועדון.
```

CODE

Final HEAD	51df79463dd65e88541a926217caef3bfa744462
Git status	clean
Source changed after 51df794	NO — not one line
Existing binaries	still valid and still traceable to the release commit
Push	not performed

PRODUCTION

- **Firestore Rules: NOT DEPLOYED.** `firestore.rules` untouched. The repo's further `isFullAccount()` tightening for `/rounds`, `/playerStats`, `/ratings`, `seasonTitles` and `/pairStats` is still waiting, as instructed.
- **No other production deploys.** No Functions, Hosting, Storage, indexes, Remote Config, App Check or Auth configuration was touched.
- **Production writes made:** the Play production track was modified twice (release created at 10%, then halted) — both intended. The QA account signed in and out. No club, game, registration or filler application was created.

What I recommend

The fix is small and squarely inside the feature this release is about. My suggestion, for your decision:

1. Fix `signOut()` so it re-enters the boot path rather than leaving a null session with no route out.
2. Add a regression test that asserts the post-sign-out state can never be `currentUser: null && !guestInitFailed && hydrated` — the same style of

cheap source-level guard as `hooksAfterEarlyReturn.test.ts` .

3. Rebuild (vc244 / build 108), re-run the short matrix, and resume the staged rollout from the halted release.

PARTIAL SUBMISSION

Google Play: submitted to production as 1.1.15 / versionCode 243 at a 10% staged rollout, then **halted** when the regression was found. 1.1.14 remains the active release.

App Store: build 107 uploaded and processed (`VALID`) in TestFlight. ****No** App Store version created and nothing submitted to App Review**, because the defect is in shared JavaScript and affects iOS identically.

Human action required: decide whether to fix the sign-out hang and rebuild (recommended), or to accept it and resume the rollout. A code change is the only route forward, and §1 forbids me from making one without stopping first — so this is your call. If you approve the fix, the next round is: one-line change, regression test, rebuild both platforms, short re-QA, resume Play from the halted release and submit iOS to review.